

Raport aplikacji Zaliczone — od A do Z

Data raportu: 6 września 2026

Repozytorium: `zaliczone`

Cel dokumentu: kompletny opis funkcjonalny i techniczny — każda podstrona, panel, zakładka, reguła biznesowa, mutacja, cache i stack.

Od poprzedniej wersji (24.08.2026): pełny pipeline rekrutacji Google Forms → panel admina, terminy testów (3 dni robocze), wyniki na kafelkach z progiem 75%.

Spis treści

1. [Czym jest Zaliczone](#)
 2. [Role użytkowników i dostęp](#)
 3. [Cykl biznesowy miesiąca](#)
 4. [Statusy lekcji i wypłat](#)
 5. [Strefa publiczna](#)
 6. [Panel nauczyciela \(TUTOR\)](#)
 7. [Panel admina \(ADMIN\)](#)
 8. [API \(Route Handlers\)](#)
 9. [Model danych \(Supabase\)](#)
 10. [Server Actions — katalog](#)
 11. [Stack technologiczny](#)
 12. [Cache i optymalizacje](#)
 13. [Integracje zewnętrzne](#)
 14. [Bezpieczeństwo](#)
 15. [Testy i seed](#)
 16. [Znane ograniczenia i uwagi](#)
-

1. Czym jest Zaliczone

Zaliczone to aplikacja webowa do zarządzania agencją korepetycji:

- **Koordynator (ADMIN)** — weryfikuje wpłaty uczniów, wypłaca nauczycieli, prowadzi księgowość (NDG/JDG), zarządza kadrą i cennikiem, **prowadzi rekrutację korepetytorów** (Google Forms → testy → zatrudnienie).
- **Korepetytor (TUTOR)** — planuje lekcje, prowadzi uczniów, zgłasza lekcje do weryfikacji, ogląda finanse i generuje ewidencję godzin do podpisu.

- **Uczeń** — **nie ma konta** w systemie. To rekord w tabeli `students` przypisany do nauczyciela (imię, przedmiot, poziom, kontakt, ewentualna blokada).
- **Kandydat** — **nie ma konta**; rekord w `candidates` z formularza rekrutacyjnego / wyników quizów.

Landing publiczny (/) pozwala znaleźć korepetytora (filtr poziom/przedmiot/dni) i zostawić zgłoszenie waitlisty e-mailem.

2. Role użytkowników i dostęp

Rola	Gdzie pracuje	Domyślne przekierowanie po logowaniu
ADMIN	/admin/*	/admin
TUTOR	/panel, /terminarz, /uczniowie, /finanse, /profil, /przewodnik	/panel

Middleware (`middleware.ts` + `lib/supabase/middleware.ts`)

1. Odświeża sesję Supabase (cookie SSR).
2. **Ścieżki publiczne** (bez logowania): `/`, `/login`, `/ustaw-haslo`, `/auth/callback`, `/regulamin`, `/polityka-prywatnosci`.
3. Brak sesji na chronionej ścieżce → `/login?next=<pathname>`.
4. Zalogowany na `/login` → `/panel` (ADMIN zaraz potem trafia na `/admin` przez regułę 5).
5. ADMIN na `/panel` → `/admin`.
6. Nie-ADMIN na `/admin*` → `/panel`.

Auth callback (`app/auth/callback/route.ts`)

- Wymiana `code` na sesję (`exchangeCodeForSession`) albo `verifyOtp` (invite / magic link).
- Parametr `next` jest sanitizowany — tylko ścieżki względne zaczynające się od `/`.
- Błąd → `/login?error=invite`.

Zaproszenie nauczyciela

Admin tworzy konto (`createTutorAccount`) → e-mail invite (Resend) → link `/auth/callback?...&next=/ustaw-haslo` → ustawienie hasła → `/panel`.

Wylogowanie

`signOut()` z `lib/data/mutations.ts` → `redirect /login`.

3. Cykl biznesowy miesiąca

Źródło terminów: `lib/dates.ts` (`DATES`).

Etap	Kiedy (w następnym miesiącu względem miesiąca pracy M)	Co się dzieje
Ewidencja godzin	od 1., deadline 3.	Nauczyciel generuje PDF ewidencji, podpisuje, odsyła
Rachunki	wysyłka 3., zwrot do 5.	Dokumenty do podpisu (proces offline + checklisty w kalendarzu admina)
Weryfikacja dokumentów	od 6.	Zgodność podpisanych dokumentów z systemem
Wypłaty	do 10.	Admin oznacza PAID + przelew bankowy + e-mail
Zamknięcie miesiąca	najwcześniej od 6., target 15.	closeMonth w księgowości
Przegląd finansów	16.	Limity NDG/VAT, koszty
Podatki	do 20.	Zaliczka PIT (głównie JDG)

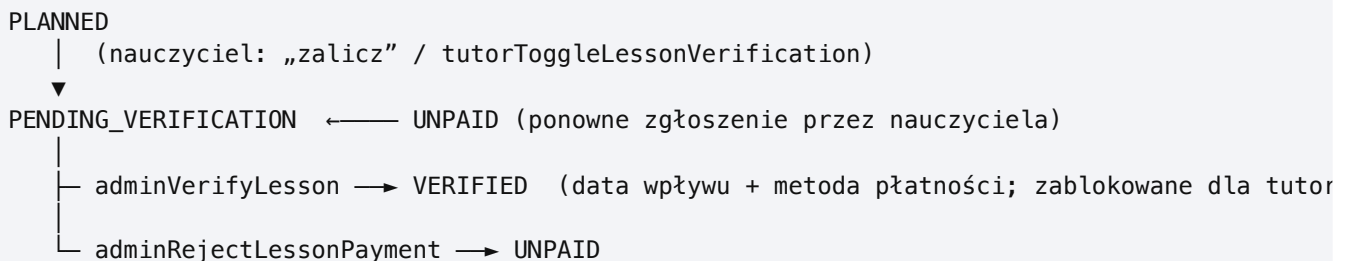
Roczne: zestawienie kosztów (15.01), PIT-11, PIT nauczycieli, PIT-36, zamknięcie roku.

Premia nauczyciela

- Progi: **40 / 50 / 60 godzin** lekcji VERIFIED → po **+100 zł** na próg (łącznie do **300 zł**). Pasek na dashboardzie pokazuje tylko aktualny segment.
- Udział bazowy tutora w stawce klienta (fallback): **70%** (`TUTOR_SHARE = 0.7`).
- W praktyce wynagrodzenie liczone jest ze **stawek pracownika z cennika** (`worker_rate_pln` × godziny) + premia.

4. Statusy lekcji i wypłat

Lekcje (`lessons.status`)



Zasady:

Kto	Może	Nie może
TUTOR	Tworzyć/edytować/usuwać <code>PLANNED</code> (nie przeszłe); przełączać <code>PLANNED↔PENDING</code> ; <code>UNPAID→PENDING</code>	Cofnąć <code>VERIFIED</code> ; mutować zamknięty miesiąc; lekcje zablokowanego ucznia; kolizje godzin
ADMIN	Zatwierdzić / oznaczyć brak wpłaty	— (w UI rozliczeń)

Alert streak: 3 lekcje z rzędu w statusach `UNPAID|VERIFIED` , wszystkie `UNPAID` → alert `UNPAID_STREAK` dla ADMIN → opcjonalna blokada ucznia → alert `STUDENT_BLOCKED` dla tutora.

Wypłaty (payouts)

- Brak rekordu = wypłata jeszcze nieoznaczona.
- `status === "PAID"` = wypłacone (UI „Wypłacone”).
- Typ `PENDING_DOCS` istnieje w typach, operacyjny flow opiera się głównie na obecności `PAID` .

Zamknięty miesiąc (closed_months)

`assertMonthOpen(monthKey)` w `lib/actions/guards.ts` blokuje mutacje (lekcje, wypłaty, koszty) dla zamkniętego miesiąca. Fail-open, jeśli tabela jeszcze nie istnieje (pre-migracja).

5. Strefa publiczna

5.1 / — Landing

Cel: marketing, matching korepetytorów, waitlista, wejście do logowania.

Dane (SSR):

- `getPublicTutorCards()` — aktywni TUTOR bez zakończonej umowy, przedmioty/poziomy, telefon, e-mail, OLX, zdjęcie.
- `getPriceTiers()` — etykiety poziomów do filtra.

UI: `LandingPageClient` .

Akcje:

- Filtr: poziom + przedmiot + dni tygodnia.
- Karty korepetytorów z kontaktem.
- Modal waitlisty → Server Action `submitTutorWaitlist` → e-mail na `COMPANY.email` (`sendTutorWaitlistEmail`). **Bez zapisu do DB** w samej akcji.
- Linki: `/login` , `/regulamin` , `/polityka-prywatnosci` .

Nie pozwala: założyć konta ucznia, zarezerwować lekcji online.

5.2 /login

- Formularz e-mail + hasło (`LoginForm`).
- Klient Supabase `signInWithPassword` + odczyt `profiles.role` → redirect (patrz §2).

- Błędne dane → komunikat (bez ujawniania szczegółów konta).
- Brak UI „zapomniałem hasła”.

5.3 /ustaw-hasło

- Pierwsze hasło po zaproszeniu (`SetPasswordForm`).
- Min. 8 znaków, zgodność pól, wymagana sesja invite.
- Action: `setInitialPassword` → update przez service role + `signInWithPassword` → `/panel`.

5.4 /regulamin i /polityka-prywatnosci

- Publiczne dokumenty MDX + `LegalDocShell`.
- Bez auth, bez mutacji.

5.5 /auth/callback

- Endpoint OAuth/invite (patrz §2).

6. Panel nauczyciela (TUTOR)

Shell: `AppShell` + `LessonCompletionProvider` (z wyjątkiem printowych `/finanse/ewidencja`).

Nawigacja główna: Główna (`/panel`), Terminarz, Uczniowie, Finanse; dodatkowo Przewodnik, Profil; wylogowanie.

Global search: `searchWorkspace` → uczniowie (`/uczniowie?student=`), lekcje (`/terminarz?lesson=`).

Guard stron: brak profilu → `/login`; ADMIN → `/admin`.

6.1 /panel — Główna nauczyciela

Cel: skrót tygodnia, premia, alerty, uczniowie, finanse miesiąca.

Dane: lekcje, uczniowie, linie VERIFIED, cennik, alerty tutora; wyliczenia godzin/wypłaty (`sumTutorPayoutFromCennik`, `premia bonusProgress`), `lessonStats`.

UI: `DashboardLayout` — `AlertsBanner`, `BonusProgressBar`, `WeeklySchedule`, `MonthlyCalendar`, `StudentsPanel`, kafelki finansowe, skrót do przewodnika.

Pozwala:

- `tutorToggleLessonVerification` — PLANNED↔PENDING, UNPAID→PENDING.
- `dismissAlert` na alertach TUTOR.

Nie pozwala: CRUD lekcji/uczniów (to Terminarz / Uczniowie); cofnąć VERIFIED; oznaczać wypłat.

6.2 /terminarz

Cel: planowanie i zarządzanie lekcjami (tydzień / miesiąc).

Dane: lekcje, uczniowie, `active_subjects`, alerty.

UI: `TerminarzPageView` — kalendarz, modal lekcji, cykliczność (once / weekly / custom), deep-link `?lesson=`.

Server Actions (`lib/actions/lessons.ts`):

Funkcja	Co robi	Blokady
<code>insertLessons</code>	Tworzy lekcje <code>PLANNED</code> (seria opcjonalna)	Data $\geq$ dziś; przedmiot z uprawnień; uczeń nie <code>blocked</code> ; brak kolizji; miesiąc otwarty
<code>updateLesson</code>	Edycja	Tylko własne <code>PLANNED</code> ; nie przeszłe; otwarty miesiąc; bez kolizji
<code>deleteLesson</code>	Usunięcie	Tylko <code>PLANNED</code> , nie przeszłe
<code>deleteLessonAndRemainingInSeries</code>	Ta + późniejsze w serii	Tylko <code>PLANNED</code>
<code>deleteLessonsByIds</code>	Masowe usuwanie	Tylko <code>PLANNED</code>
<code>tutorToggleLessonVerification</code>	Zgłoszenie do weryfikacji / cofnięcie	<code>VERIFIED</code> locked; zamknięty miesiąc

Pomocnicze: `lessonDatesFromDraft`, `assertNoTutorTimeConflict / overlap [start, end)`.

6.3 /uczniowie

Cel: baza uczniów nauczyciela.

UI: lista, sortowanie, filtr przedmiotu, modal add/edit, historia lekcji, badge `blocked` , deep-link `?student=`.

Actions (`lib/actions/students.ts`):

Funkcja	Zasady
<code>insertStudent</code>	Przedmioty/poziom z <code>active_subjects</code> ; stawka z <code>price_tiers</code>
<code>updateStudent</code>	Zablokowany $\rightarrow$ błąd („możesz go usunąć”); te same reguły uprawnień
<code>deleteStudent</code>	Tylko własni (<code>tutor_id</code>)

Nie pozwala: edytować zablokowanego ucznia; dodawać lekcji dla zablokowanego (też w `insertLessons`).

6.4 /finanse

Cel: podgląd wynagrodzenia i lekcji `VERIFIED` (read-mostly).

Dane: linie `VERIFIED`, payouts, closed months, cennik, liczba uczniów.

UI (`FinanseClient`):

- Wybór miesiąca (zamknięte oznaczone).
- Modal cennika (tylko odczyt).
- „Do wypłaty” + premia + badge gdy `PAID` .

- Lista zatwierdzonych lekcji z udziałem nauczyciela.
- Link **Generuj ewidencję** → `/finanse/ewidencja?month=YYYY-MM` gdy są VERIFIED i PDF odblokowany (`isEwidencjaPdfAvailable` — od 1. dnia kolejnego miesiąca).

Nie pozwala: oznaczać wypłat, zmieniać statusów lekcji, edytować cennika.

6.4b `/finanse/ewidencja`

- Widok do druku (bez AppShell).
 - Warunki: poprawny `month`, PDF dostępny, ≥ 1 lekcja VERIFIED.
 - `EwidencjaPrintView` — dokument offline (podpis + mail do koordynatora — opis w przewodniku).
-

6.5 `/profil`

Cel: dostępność, przedmioty, dokumenty Drive (odczyt).

Pozwala:

- `setAcceptingStudents` — sync alertu `STOP_TEACHING`.
- `insertSubjectRequest` — wniosek o nowy przedmiot/poziom (`PENDING`).
- Przeglądanie i pobieranie plików Drive (`getTutorDriveFilesForViewer`, `GET /api/drive/files/[fileId]`).

Nie pozwala (tylko admin): edycja imienia, telefonu, OLX, konta bankowego, PIT, hasła (poza ścieżką `invite`), upload do Drive z UI profilu.

6.6 `/przewodnik`

- Statyczna instrukcja: terminy `DATES`, dane firmy do faktur, placeholder wideo.
- Bez mutacji.

6.7 `/kalendarz`

- **Redirect** → `/terminarz` (alias historyczny).

6.8 `/powiadomienia`

- **Redirect** → `/panel`.
 - Infrastruktura (`notifications`, `markNotificationRead`, `NotificationsList`) istnieje, ale **nie jest podpięta** pod osobną trasę nauczyciela. Nauczyciel widzi głównie **alerty**.
-

7. Panel admina (ADMIN)

Layout: `app/admin/layout.tsx` + `AdminLayoutClient`.

Sidebar: Główna, Kalendarz, Lekcje, Wypłaty, Księgowość, Nauczyciele, **Rekrutacja**, Cennik i przedmioty, wylogowanie.

Global search: nauczyciele, lekcje, strony admina (`searchWorkspace`).

Bez shella (print): `/admin/ksiegowosc/ewidencja` , `/admin/ksiegowosc/koszty` ,
`/admin/wypłaty/lista-plac` .

Badge alertów na „Główna”: `getOpenAdminAlerts()` .

7.1 `/admin` — Główna (dashboard)

Cel: pulpit koordynatora — finanse miesiąca, zespół, lekcje, alerty, mini-cykl miesięczny.

Finanse miesiąca (kafelki):

Kafelek	Znaczenie
Przychód	Suma kwot klienta z lekcji <code>VERIFIED</code> w bieżącym miesiącu
Koszty	Koszt tutorów (cennik + premia) + koszty operacyjne miesiąca
Dochód	Przychód – koszty
Nieopłacone	Suma linii <code>UNPAID</code> w miesiącu

Podział (donut) — w sekcji „Zespół i lekcje” (w miejscu dawnych „Umowy - 30 dni”): Koszty / Dochód / Nieopłacone — w tej samej skali wizualnej co donut „Lecje”.

Lecje (donut): Do zatwierdzenia / Zatwierdzone / Nieopłacone (liczby).

Pozostałe: liczba nauczycieli i uczniów; kalendarz bieżącego miesiąca + checklista terminów cyklu (`localStorage`, bez serwera); `AlertsBanner` (`blockStudentFromAlert` , `dismissAlert`).

Źródła danych: `getCachedCoordinatorDashboardLines` , `getAllOperatingExpenses` ,
`getAllTutorProfiles` , `count students` , `getPriceTiers` , `getOpenAdminAlerts` .

7.2 `/admin/rozliczenia` — Lecje

Cel: weryfikacja wpłat uczniów w układzie tygodniowym — 3 kolumny.

Kolumna	Status	Akcje
Do zatwierdzenia	<code>PENDING_VERIFICATION</code>	Zatwierdź / Brak wpłaty
Zatwierdzone	<code>VERIFIED</code>	Tylko podgląd (data wpływu zablokowana)
Nieopłacone	<code>UNPAID</code>	Ponowne zatwierdzenie / obsługa

Filtry: tydzień (`WeekNavigator`), wyszukiwanie tekstowe, deep-link `?q=&date=` .

Actions:

- `adminVerifyLesson(lessonId, paymentReceivedAt, paymentMethod?)` — wymaga metody: Przelew tradycyjny / BLIK / Przelewy24 / Gotówka; `syncUnpaidStreakAlert` ; `cache bust` .
- `adminRejectLessonPayment(lessonId)` → `UNPAID` .

Blokady: `assertMonthOpen` po dacie lekcji; po `VERIFIED` data wpływu tylko do odczytu.

UI: `ConfirmDialog`, animacje `pulse/hop`, `WeekVerificationBar`. Karty zatwierdzonych lekcji: wyraźniejszy zielony border (`border-2 border-moss/70`), zaokrąglenie `rounded-app` (nie pełny pill).

7.3 `/admin/wypłaty` — Wypłaty

Cel: lista płac za **miesiąc pracy = poprzedni** względem wybranego „miesiąca wypłaty” (`previousMonthKey`). Tylko lekcje `VERIFIED`.

Per nauczyciel: lekcje, godziny, wynagrodzenie (cennik), premia, razem, status `PAID`, nr konta.

Kafelki: Przychód (klient) / Do wypłaty · Wypłacone / Marża agencji.

Actions:

UI	Action
Oznacz jako wypłacone	<code>markPayoutPaid</code> — <code>upsert payouts</code> + <code>sendPayoutConfirmationEmail</code> ; kwotę można skorygować w dialogu
Odznacz	<code>unmarkPayoutPaid</code>
PDF	<code>/admin/wypłaty/lista-płac?month=...</code>

Blokady: `assertMonthOpen(workMonthKey)`.

`/admin/wypłaty/lista-płac`

- Dokument do druku/PDF przeglądarki (umowy zlecenia).
 - Guard `ADMIN`. Bez mutacji DB.
 - **Uwaga:** PDF może liczyć udział `TUTOR_SHARE` (70%), podczas gdy tabela wypłat używa stawek z cennika — możliwe drobne różnice.
-

7.4 `/admin/ksiegowosc` — Księgowość

Cel: ewidencja sprzedaży, koszty, P&L, limity `NDG/VAT`, podatki/`ZUS`, zamknięcie miesiąca, przełączenie `NDG→JDG`.

Query: `?month=YYYY-MM`.

Przełączniki UI:

- **NDG / JDG** — tekst przy tytule „Księgowość” (`LegalModeInlineSwitch`); aktywny granatowy, nieaktywny wyblakły; zawsze widoczny. Przejście na `JDG`: `switchToJDG()` — **nieodwracalne**.
- **Księgowość miesięczna / roczna** — zawsze rozwinięte, pełna szerokość.

Dane: lekcje, `payouts`, `closed months`, `operating expenses`, `business settings`, `price tiers`.

Sekcje

1. **Kafelki KPI** — m.in. przychód, wypłaty, dochód, liczba lekcji zatwierdzonych (kafelek szary).

2. Limity

- `NDG`: pasek kwartalny (`NDG_QUARTERLY_LIMIT` $\approx$ 10 813,5 zł), alert `CEIDG` 7 dni.
- `JDG`: pasek `VAT` roczny (`VAT_ANNUAL_LIMIT` 240 000).

- 1. **Ewidencja sprzedaży** — cała sekcja w ramce (`card-quiet`): tabela VERIFIED + przycisk PDF.
- 2. **Koszty** — cała sekcja w ramce: nagłówek „Koszty”, podpis okresu, kompaktowy formularz (data, nr, opis, wystawca, kwota, plik w jednej linii), tabela faktur, PDF.
- `createOperatingExpense` / `deleteOperatingExpense` .
- Załącznik: Google Drive (`Faktury/YYYY/miesiąc YYYY/`) lub fallback Supabase Storage; max ~12 MB.
- 1. **Podsumowanie miesiąca** — tabela (tło `rgb(245,248,255)` , border granatowy); kolumna **Dochód** (dawniej Marża); bez zbędnego podtytułu.
- 2. **Rozliczenia – Zrób to sam** — 2 kolumny równej wysokości:
- **Podatki** (ramka) — NDG: zera + kwota wolna; JDG: zaliczka PIT, ZUS, zdrowotna, etap Ulga na start / Preferencyjny.
- **Podsumowanie** (neon `card-feature`) — przychód minus wypłaty/podatki/ZUS/koszty → zysk/strata na rękę (wiersz wyniku przyklejony do dołu karty).
- 1. **Zamykanie miesiąca** — checklista + checkbox zgodności z kontem bankowym + `closeMonth` .

Reguły zamknięcia

- 1. Najwcześniej od **6.** dnia następnego miesiąca.
- 2. Brak lekcji `PLANNED` / `PENDING_VERIFICATION` .
- 3. Wszystkie wypłaty miesiąca `PAID` .
- 4. Checkbox bankowy — **tylko UI** (nie walidowany na serwerze).
- 5. Po zamknięciu: brak dodawania/usuwania kosztów; mutacje lekcji/wypłat zablokowane przez `assertMonthOpen` .

NDG vs JDG (skrót)

	NDG	JDG
PIT miesięczny	0 (PIT-36 rocznie)	zaliczka ~12% – ulga
ZUS właściciela	0	Ulga na start (0 społeczne, 6 mies.) lub preferencyjny
Zdrowotna	0	9% dochodu, min ~432,54
Limit	kwartalny NDG	roczny VAT

Logika: `lib/podatki.ts` , `lib/podatki-config.ts` .

Printy

Trasa	Zawartość
<code>`/admin/ksiegowosc/ewidencja?month`</code>	<code>year=`</code> Ewidencja sprzedaży VERIFIED
<code>`/admin/ksiegowosc/koszty?month`</code>	<code>year=`</code> Zestawienie kosztów (przy roku: wypłaty PAID + koszty)

Wydruki = React + CSS print / „Zapisz jako PDF” w przeglądarce (bez Puppeteer).

7.5 /admin/nauczyciele

Cel: lista kadry — zakładki **Aktywni / Byli** (`contract_end ≤ dziś`).

Akcje:

- `createTutorAccount` — Auth invite, profil TUTOR, folder Drive, welcome e-mail, opcjonalnie zdjęcie.
- Karty: uczniowie, lekcje/godziny miesiąca, `BonusProgressBar`, link do profilu.

/admin/nauczyciele/[id] — profil nauczyciela

Dane: podsumowanie, uczniowie, lekcje miesiąca (`pending/unpaid/verified` — **tylko odczyt**), panel PIT.

Side effects: `ensureTutorRootFolder` / `ensureTutorDriveFolder` (foldery w tle).

Actions (m.in.):

Action	Opis
<code>updateTutorProfile</code>	Edycja; rename/move folderu Drive przy zmianie nazwy/końca umowy
<code>uploadTutorPhoto</code> / <code>clearTutorPhoto</code>	Max 5 MB, JPG/PNG/WebP
<code>archiveTutorAccount</code>	<code>contract_end</code> , ban Auth, folder → „byli”
<code>unblockStudent</code>	Odblokowanie ucznia
<code>updateTutorPitIdentity</code> / <code>updateTutorTaxYearEntry</code>	PESEL, US, NIP, koszty/zaliczki/ZUS/ulga młodych; przychód z PAID

7.6 /admin/cennik

Cel: stawki poziomów + wnioski o przedmioty.

Tabela: poziom / klient płaci / nauczyciel dostaje / marża agencji.

Actions: `savePriceTiers` (delete-all + insert), `approveSubjectRequest`, `rejectSubjectRequest`.

Efekt approve: aktualizacja `active_subjects` nauczyciela + cache subjects/cennik.

7.7 /admin/kalendarz

Cel: kalendarz operacyjny + checklisty (Nowy pracownik, Wypłata, własne terminy).

Stan: `localStorage` — bez bazy.

Terminy: generowane z DATES (ewidencje, rachunki, weryfikacja, wypłaty, zamknięcie, limity, podatki, terminy roczne).

UI: statusy todo/doing/done, podzadania, nawigacja miesiący.

7.8 /admin/rekrutacja — Rekrutacja korepetytorów

Cel: pipeline od zgłoszenia Google Forms, przez wysyłkę / wyniki testów, po zatrudnienie lub odrzucenie.

Źródło danych: tabela `candidates` (webhook Forms + edycja w panelu). Dokumentacja skryptów: `docs/google-forms-apps-script.md`.

Flow operacyjny

```
Google Form (APPLICATION)
  | Apps Script → POST /api/webhooks/google-forms
  ▼
Karta kandydata (NEW)
  | Admin: odpowiedź testów → wysyłka e-mailem (poza panelem)
  | checkbox „Test wysłany” + data wysyłki
  ▼
IN_PROGRESS (+ termin 3 dni roboczych)
  | Quizy Google Forms (TEST_RESULT) dopinają wyniki po e-mailu
  ▼
Hire → createTutorAccount | Reject → e-mail + status REJECTED
```

Kolejność reguł: najpierw zgłoszenie APPLICATION tworzy kartę. Sam wynik quizu **nie** powinien być jedynym źródłem pełnego profilu (stąd możliwy historycznie; produkcyjnie APPLICATION-first).

Statusy kandydata

Status	Znaczenie
NEW	Nowe zgłoszenie / jeszcze bez wysłanych testów
IN_PROGRESS	W procesie (testy wysłane lub w toku)
HIRED	Zatrudniony (konto TUTOR)
REJECTED	Odrzucony (backfill APPLICATION nie tworzy duplikatu — <code>updated_closed</code>)

Grupy workflow (otwarte kandydatury)

Bucket	Warunek (skrót)
Po czasie	Testy wysłane, nie wszystkie oddane, po terminie
Test nie wysłany	Brak <code>test_sent_manually</code> / <code>test_sent_at</code>
W toku	Wysłane, w oknie 3 dni roboczych
Zrobione	Wszystkie oczekiwane przedmioty mają wynik

Logika: `getCandidateTestWorkflow` / `testResponseWindow` w `lib/recruitment/test-links.ts`.

Termin na testy (3 dni robocze)

- Start zegara = pierwszy **dzień roboczy** w dniu wysyłki lub później (**sob/nd** → **poniedziałek**).
- Termin = **3. dzień roboczy włącznie** (np. poniedziałek → środa).
- Badge w profilu: „jeszcze X dni robocze” — $X \leq 3$.

UI kafelków i profilu

- Przedmioty i poziomy** — wszystkie zaznaczenia z CheckboxGrid (per przedmiot).
- Testy do wysłania** — jeden test na przedmiot = **najwyższy** zaznaczony poziom; Matura w podpowiedzi mapuje się na SS rozszerzony (gdy brak osobnego Forms).

- Chip na liście: Niemiecki 15/15 (dopasowanie po znormalizowanej nazwie, np. Język niemiecki ↔ Niemiecki).
- Wynik < 75% → czerwony (kafelek + profil); ≥ 75% → zielony.
- Checkboxy: **Test wysłany** (+ data), **Testy sprawdzone samodzielnie**.
- Hire / Reject z potwierdzeniem.

CheckboxGrid i „Inne”

Forms zwraca siatkę jako tablicę poziomów **po indeksie wiersza**. Apps Script mapuje wiersze (Język polski ... Inne) i czyta tekst z opcji Other przy pytaniu „**Jakiego przedmiotu chcesz uczyć?**” (np. hiszpański, fotografia).

Actions (lib/actions/recruitment.ts)

Action	Opis
setCandidateTestsSent	Flaga + test_sent_at
setCandidateTestsReviewed	Flaga przeglądu wyników
setCandidateStatus	Zmiana statusu
hireCandidate	Status HIRED + createTutorAccount (przedmioty z required_tests)
rejectCandidate	Status REJECTED + e-mail odrzucenia (Resend)

8. API (Route Handlers)

Endpoint	Dostęp	Cel
GET /api/drive/files/[fileId]?tutorId=	TUTOR: własny folder; ADMIN: wymaga tutorId	Strumień pliku z folderu nauczyciela (Docs → PDF)
GET /api/drive/invoices/[fileId]	tylko ADMIN	Strumień faktury z drzewa Faktury
POST /api/webhooks/google-forms	Header x-webhook-secret = GOOGLE_FORMS_WEBHOOK_SECRET	APPLICATION upsert kandydata; TEST_RESULT dopina wynik do karty po e-mailu

Cache-Control: private, no-store (Drive). Webhook: JSON { type, data|... }; bez poprawnego sekretu → 401.

9. Model danych (Supabase)

Część migracji leży w repo: supabase/migrations/ (0002 ... 0020). Uruchamiane ręcznie w SQL Editorze Supabase; kod ma fallbacki przy braku kolumn.

Tabele (skrót)

Tabela	Rola
profiles	1:1 z Auth; rola, przedmioty, kontakt, bank, Drive, photo, PIT JSON
students	Uczniowie tutora; poziom, stawka, blocked, kontakt
lessons	Harmonogram + status + payment_received_at/method + series_id
price_tiers	Cennik poziomów
payouts	Wypłaty miesięczne + meta (lekcje, premia, count)
subject_requests	Wnioski o przedmioty
closed_months	Zamknięte miesiące księgowe
business_settings	Singleton NDG/JDG + data rejestracji JDG
operating_expenses	Koszty + załączniki
document_folders / document_files	Drzewo dokumentów (Storage)
alerts	UNPAID_STREAK, STOP_TEACHING, STUDENT_BLOCKED
notifications	EWIDENCJA_REQUEST, CENNIK_UPDATE, PAYOUT, INFO
candidates	Rekrutacja: zgłoszenie Forms, poziomy, wyniki testów, status

candidates (migracje 0018–0020)

Kolumna	Opis			
full_name, email, phone, dob	Dane osobowe			
student_status, university, experience	Profil studenta / doświadczenie			
required_tests	JSONB: lista { subject, level } (wszystkie zaznaczenia)			
levels	Notatka tekstowa „Przedmiot: poziomy...”			
hours_per_week, cv_url	Dostępność / CV			
tests_expected / tests_completed	Liczniki (expected ≈ unikalne przedmioty)			
test_results	JSONB: lista { subject, level, score } (wiele poziomów OK)			
test_sent_manually, test_sent_at	Wysyłka testów + data startu terminu			
tests_reviewed_manually	Admin sprawdził wyniki			
status	NEW \	IN_PROGRESS \	REJECTED \	HIRED

RLS: tylko `ADMIN` (select/insert/update/delete). Webhook używa **service role**.

Storage buckets

- `documents` — dokumenty + legacy załączniki kosztów.
- `tutor-photos` — publiczne URL zdjęć (landing).

Rozróżnienie załączników kosztów

- `attachment_path` **bez** `/` = Google Drive fileId.
- `attachment_path` **ze** `/` = ścieżka Supabase Storage.

10. Server Actions — katalog

Admin (`lib/actions/admin.ts`)

```
uploadTutorPhoto, clearTutorPhoto
createTutorAccount, updateTutorProfile, updateTutorContactFields
archiveTutorAccount, deleteTutorAccount (= archive)
updateTutorPitIdentity, updateTutorTaxYearEntry
savePriceTiers
approveSubjectRequest, rejectSubjectRequest
closeMonth, switchToJDG
markPayoutPaid, unmarkPayoutPaid
adminVerifyLesson, adminRejectLessonPayment
createOperatingExpense, deleteOperatingExpense
```

Rekrutacja (`lib/actions/recruitment.ts`)

```
setCandidateTestsSent, setCandidateTestsReviewed
setCandidateStatus, hireCandidate, rejectCandidate
```

Tutor / wspólne

Moduł	Funkcje
lessons.ts	insert/update/delete (serie), tutorToggleLessonVerification
students.ts	insert/update/delete student, insertSubjectRequest
profile.ts	setAcceptingStudents, setInitialPassword
alerts.ts	dismissAlert, blockStudentFromAlert, unblockStudent, sync streak/stop
notifications.ts	markNotificationRead
drive.ts	list/ensure/sync folderów (viewer)
documents.ts	signed URL, ensure folders
search.ts	searchWorkspace
waitlist.ts	submitTutorWaitlist
guards.ts	assertMonthOpen, monthKeyFromDate
mutations.ts (client)	signOut, helpery dat draftu

11. Stack technologiczny

Warstwa	Technologia
Framework	Next.js 16.1.6 (App Router)
UI	React 19.2.3
Język	TypeScript 5 (strict)
Pakiety	pnpm 10.19, Node ≥ 20
Baza / Auth / Storage	Supabase (@supabase/ssr, @supabase/supabase-js)
Styling	Tailwind CSS 4 + custom theme (app/globals.css)
Fonty	Poppins (logo), Space Grotesk + IBM Plex Mono (panel finansowy)
E-mail	Resend
Pliki	Google Drive API (googleapis) — Shared Drive
Dokumenty prawne	MDX (@next/mdx)
Testy	Vitest 2.1.9
Lint	ESLint 9 + eslint-config-next
Dev	next dev (Turbopack domyślnie w Next 16; dev:clean czyści .next)

Konfiguracja: experimental.serverActions.bodySizeLimit: "6mb" (zdjęcia ≤ 5 MB).

React Compiler: peer opcjonalny Nexta — **nie jest jawnie włączony** w next.config.ts.

12. Cache i optymalizacje

12.1 Strategia cache (lib/cache.ts)

Mechanizm	Kiedy	Zachowanie
bustTag → updateTag	Mutacje krytyczne (Zalicz, wypłata, cennik, hasło, koszty...)	Natychmiastowe unieważnienie — read-your-own-writes
staleTag → revalidateTag(tag, "max")	KPI dashboardu	Dopuszczalna chwilowa nieświeżość
revalidatePath	Po mutacjach	Odświeżenie konkretnych tras RSC
bustLessonAndBonus	Zmiana lekcji	Tagi lessons-{id}, bonus-{id}, TAG.lessons, opcjonalnie finance-{month}
revalidateLessonPages	Po lekcjach	/panel, /terminarz, /admin/rozliczenia, /finanse

Tagi stałe (TAG): lessons, finance, accounting, cennik, notifications, documents, dashboard-stats, public-tutors.

Tagi per encja: lessons-, students-, subjects-, bonus-, finance-, payouts-, notifications-, documents-.

12.2 unstable_cache (lib/data/queries.ts)

Query	TTL	Tagi (skrót)
getTutorStudents	revalidate: false (tylko tagi)	students + lessons
getTutorVerifiedFinanceLines	30 s	lessons, bonus, finance
getCachedCoordinatorDashboardLines	60 s	dashboard-stats, lessons
getAdminTutorSummaries	30 s	lessons, dashboard, cennik, finance
getPriceTiers / getClosedMonths / subjects / documents / notifications	false + tagi	odpowiednie TAG

Dodatkowo: **React.cache** na getCurrentUserProfile i części list lekcji — deduplikacja w jednym requestcie RSC.

12.3 Inne optymalizacje

- Promise.all** na niemal wszystkich page RSC (równoległe fetchowanie).
- Graceful degradation** — brak tabel/kolumn (kody PostgREST) → fallback / komunikat migracji zamiast twardego crasha.
- Drive streaming** — Readable → Web Streams, bez ładowania całego pliku do pamięci tam gdzie to możliwe.

4. **Resend opcjonalny** — brak `RESEND_API_KEY` → log warning, app działa.
5. **Drive opcjonalny** — koszty lecą do Storage; foldery nauczycieli tworzone gdy skonfigurowane.
6. **Print PDF w przeglądarce** — zero ciężkiego silnika PDF po stronie serwera.
7. **Instrumentation** — tłumienie znanych warningów Node.
8. **Obrazy** — `next/image` + `remotePatterns` (Supabase, Unsplash).
9. **localStorage** na checklistach kalendarza admina — zero roundtripów dla UI-only stanu.

12.4 Co NIE jest cache'owane agresywnie

- `getPublicTutorCards` na landingu — świeże odczyty (service role) w `Promise.all` z cennikiem.
 - Wiele mutacji admina celowo robi szeroki `revalidatePath`, żeby UI od razu pokazywał stan po Zalicz / wypłacie / zamknięciu.
-

13. Integracje zewnętrzne

Google Forms + Apps Script (rekrutacja)

- Formularz główny (**APPLICATION**) + osobne quizy przedmiotów (**TEST_RESULT**).
- Skrypt w `docs/google-forms-apps-script.md`: `onFormSubmit`, `backfillExistingApplications`, `backfillExistingTestResults`.
- Endpoint: `https://www.zaliczone.edu.pl/api/webhooks/google-forms`.
- Env: `GOOGLE_FORMS_WEBHOOK_SECRET` (Vercel + Apps Script `WEBHOOK_SECRET`).
- `CheckboxGrid`: mapowanie indeks → przedmiot; Other „Inne:” z pytania o przedmiot.

Google Drive — nauczyciele

- Root: `GOOGLE_DRIVE_TEACHERS_FOLDER_ID`.
- Per tutor: folder + `profiles.drive_folder_id`.
- Archiwizacja → folder „byli” (`FORMER_TEACHERS_FOLDER_ID` / fallbacki wyszukiwania).
- Skrypt: `scripts/sync-drive-folders.mjs`.

Google Drive — faktury

- Struktura: `Faktury / {rok} / {miesiąc} {rok} /` (polskie nazwy miesiącey).
- Service account wymaga **Shared Drive** (quota My Drive = 0 dla SA).
- Env: `GOOGLE_DRIVE_INVOICES_FOLDER_ID` (+ credentials SA).

Resend

- From: marka Zaliczone / `kontakt@zaliczone.edu.pl`.
- Maile: welcome invite, potwierdzenie wypłaty, waitlista z landingu, **odrzućenie kandydata** (`sendRecruitmentRejectionEmail`).

Firma (lib/company.ts)

- Dane do faktur / przewodnika / waitlisty (nazwa, NIP, e-mail, adres).

14. Bezpieczeństwo

1. **Middleware** — rozdział ADMIN/TUTOR + publiczne ścieżki.
2. **Server Actions** — requireAdmin / sprawdzenie tutor_id = user.id.
3. **Service role** — świadomie używany do KPI, admin mutations, webhook rekrutacji i omijania edge-case RLS.
4. **API Drive** — asercja przynależności pliku do folderu tutora / drzewa faktur.
5. **Signed URL** dokumentów Storage — krótki TTL (~120 s).
6. **Sanityzacja next** w auth callback.
7. **Webhook Google Forms** — wymaga GOOGLE_FORMS_WEBHOOK_SECRET w Production.
8. **Zamknięty miesiąc** — twarda blokada mutacji finansowych/lekcyjnych.
9. **RLS** — zakładane w Supabase; app nie polega wyłącznie na RLS (defense in depth przez actions + middleware).

15. Testy i seed

Testy Vitest (lib/**/*test.ts)

- lib/podatki.test.ts — zaliczki, limity, CEIDG.
- lib/alerts/unpaid-streak.test.ts — 3x UNPAID.
- lib/dates.test.ts — daty / bonus.

Brak E2E w repo.

Seedy (scripts/ , --env-file=.env.local)

Skrypt	Cel
seed-users.mjs	konta admin/teacher
seed-battle.mjs	multi-tutor battle-test
seed-demo.mjs	demo lekcji
seed-july21.mjs	stan „środek miesiąca”
seed-clean.mjs	czyszczenie danych seed
seed-tutor-pit.mjs	poła PIT
sync-drive-folders.mjs	sync folderów Drive

16. Znane ograniczenia i uwagi

1. **Brak roli STUDENT** — uczniowie to dane, nie loginy.
 2. **/powiadomienia** — redirect; inbox powiadomień niepodpięty do UI nauczyciela.
 3. **/kalendarz (tutor)** — alias → terminarz.
 4. **Migracje SQL** — pliki w `supabase/migrations/`, ale nadal uruchamiane ręcznie w Supabase (brak automatycznego CI migrate).
 5. **Lista płac PDF vs tabela wypłat** — możliwe różnice 70% vs stawki cennika.
 6. **Checkbox „zgodność z kontem” przy zamknięciu miesiąca** — tylko UI.
 7. **Przełączenie NDG→JDG** — nieodwracalne w aplikacji.
 8. **Waitlista** — e-mail, bez trwałego rekordu DB w akcji.
 9. **React Compiler** — nie włączony jawnie.
 10. **Umowy kończące się ≤30 dni** — usunięte z dashboardu admina (zastąpione donutem Podział).
 11. **Rekrutacja — nie wszystkie przedmioty mają Forms** (np. Historia, Geografia, hiszpański) — odpowiedź bez URL.
 12. **Wysyłka linków do testów** — poza panelem (ręczny e-mail); panel tylko oznacza „wysłane” + datę.
 13. **Backfill Forms** — wymaga aktualnego Apps Script w każdym formularzu; quizy muszą mieć oceny „natychmiast” lub arkusz Score.
-

Mapa plików kluczowych

app/	
page.tsx	Landing
login/, ustaw-haslo/, auth/	Auth
panel/, terminarz/, uczniowie/	Panel TUTOR
finanse/, profil/, przewodnik/	
admin/	Panel ADMIN
page.tsx	Dashboard
rozliczenia/	Lekcje / weryfikacja
wplyaty/ (+ lista-plac)	Wpłaty
ksiegowosc/ (+ ewidencja, koszty)	
nauczyciele/ (+ [id])	
rekrutacja/	Pipeline kandydatów
cennik/, kalendarz/	
api/	
drive/	Stream plików
webhooks/google-forms/	APPLICATION + TEST_RESULT
lib/	
actions/	Server Actions (+ recruitment.ts)
recruitment/test-links.ts	Poziomy, terminy, chipy, sugestie testów
data/queries.ts, mutations.ts, mappers.ts	
cache.ts	Tagi cache
podatki*.ts, dates.ts, guards.ts	
google-drive/	Drive helpers
emails/	Resend
supabase/	clients + middleware
supabase/migrations/	SQL 0002...0020 (m.in. candidates)
docs/google-forms-apps-script.md	Canonical Apps Script
components/	UI współdzielone

Podsumowanie jednolinijkowe

Zaliczone to Next.js 16 + React 19 + Supabase + Drive + Resend + Google Forms: publiczny matching korepetytorów, panel nauczyciela (lekcje → weryfikacja), panel admina (rozliczenia → wypłaty → księgowość NDG/JDG + **rekrutacja Forms→testy→hire**), z dwupoziomowym cache i graceful degradation przy braku migracji/integracji.

Koniec raportu.

Koniec raportu. Dokument wygenerowany na podstawie stanu kodu w repozytorium (sierpień 2026).